

Vasic-Digital-Reusable-Module-Suite

एक बार बनाओ, हर जगह उपयोग करो — छोटे, अलग-थलग, स्वतंत्र रूप से परीक्षित Go और KMP मॉड्यूल का एक बेड़ा।

सारांश

digital.vasic.* (Go) और Kotlin Multiplatform नेमस्पेस के तहत प्रकाशित जेनेरिक, पुनः उपयोगी मॉड्यूल का एक बड़ा परिवार। हर मॉड्यूल स्वतंत्र है, स्वतंत्र रूप से परीक्षित और संस्करणबद्ध है, और बड़े उत्पादों (Catalogizer, HelixAgent और व्यापक बेड़े) द्वारा समान-कोडबेस सबमॉड्यूल के रूप में उपयोग किया जाता है। यह पृष्ठ उन कई छोटे उपयोगिताओं को एकीकृत करता है जो व्यक्तिगत पृष्ठों के रूप में शोर बन जातीं।

संक्षिप्त विवरण

अलग-थलग digital.vasic.* मॉड्यूल का एक चयनित संग्रह — इंफ्रास्ट्रक्चर प्रिमिटिव्स (प्रमाणीकरण, कैश, डेटाबेस, कॉन्फिगरेशन, ऑब्जर्वेबिलिटी), AI/एजेंट निर्माण खंड (RAG, वेक्टरडीबी, एम्बेडिंग्स, MCP, एजेंटिक, प्लानिंग), और रक्षात्मक-LLM गार्डरेल्स (रेडटीम, नॉर्मलाइज़) — साथ ही Kotlin Multiplatform मिरर सेट। प्रत्येक जेनेरिक, परीक्षित और पुनः उपयोगी है।

विस्तृत विवरण

vasic-digital संगठन एक संरचनात्मक दांव पर चलता है: “संविधान + कई अलग-थलग पुनः उपयोगी सबमॉड्यूल” का दर्शन, जिसमें जेनेरिक कार्यक्षमता कभी दोबारा नहीं लिखी जाती। मोनोलिथ्स के बजाय, हर पुनः उपयोगी चिंता को अपने छोटे मॉड्यूल में निकाला जाता है — अपना रिपॉजिटरी, अपने परीक्षण, अपनी दस्तावेज़ीकरण — और सख्ती से अलग रखा जाता है ताकि किसी उपभोक्ता की विशिष्टताएँ कभी अंदर न आएँ। यह पृष्ठ उन्हें समूहबद्ध करता है क्योंकि एक-एक करके देखें तो हर मॉड्यूल लाइब्रेरी-स्तर का होता है और व्यक्तिगत उत्पाद पृष्ठ के रूप में शोर बन जाता। एक साथ लेने पर, ये संगठन का वास्तविक बल गुणक हैं: एक निजी इंजीनियरिंग संपत्ति जो “नया उत्पाद बनाओ” को “सिद्ध भागों को जोड़ो” में बदल देती है, और इस दावे का ठोस आधार कि यह बेड़ा पहिया को फिर से नहीं बनाता — यह एक बहुत अच्छा पहिया बनाए रखता है और उसे हर जगह घुमाता है।

इस संग्रह में तीन समूह शामिल हैं। **इंफ्रास्ट्रक्चर प्रिमिटिव्स (Go)** हर सेवा के लिए आवश्यक प्लंबिंग प्रदान करते हैं: auth (JWT/bcrypt), cache (Redis/TTL), database (माइग्रेशन, दोहरा SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (WebSocket हब), eventbus, filesystem (बहु-प्रोटोकॉल), discovery/mdns, http3, recovery, concurrency, lazy, और अन्य। **AI/एजेंट निर्माण खंड (Go)** AI प्रणालियों के लिए आधार प्रदान करते हैं: rag, vectordb, embeddings, memory, conversation (असीमित-संदर्भ संपीड़न, इवेंट सोर्सिंग), mcp (Model Context Protocol), toolschema, skillregistry, agentic (ग्राफ़-आधारित वर्कफ़्लो ऑर्केस्ट्रेशन), planning (HiPlan/MCTS/ट्री-ऑफ़-थॉट्स), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (इनाम मॉडलिंग/RLHF), और toon (टोकन-ओरिएंटेड ऑब्जेक्ट नोटेशन)। **रक्षात्मक-LLM गार्डरेल्स** प्रतिकूल-स्थिरता उपकरण प्रदान करते हैं: RedTeam (YAML-चालित प्रतिकूल फिक्स्चर), Normalize (प्रतिकूल-इनपुट कैनोनिकलाइज़ेशन)। एक समानांतर **Kotlin Multiplatform** सेट मुख्य मॉड्यूल्स (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP आदि) को क्रॉस-प्लेटफ़ॉर्म ऐप्स के लिए मिरर करता है।

हमने इसे क्यों बनाया

हर बार शुरुआत से कई उत्पादों (Catalogizer, HelixAgent, Herald और अन्य) को विकसित करना संसाधनों की बर्बादी और असंगति का कारण बनता है। हर सामान्य चिंता को एक अलग, परीक्षित मॉड्यूल में बदल देने से सुधार और फिक्स पूरे सिस्टम में फैल जाते हैं, और हर नया उत्पाद सिद्ध भागों से मिलकर बनता है।

यह क्यों क्रांतिकारी है

वास्तव में, यह AI-केंद्रित बैकएंड बनाने के लिए एक निजी “मानक लाइब्रेरी” है—वह परत जिसे ज्यादातर टीमों इसलिए नहीं बना पाती क्योंकि वे ऑथ, कैशिंग और RAG जैसी बुनियादी संरचनाओं को पाँचवीं बार फिर से हल करने में व्यस्त रहती हैं। यहाँ इंफ्रास्ट्रक्चर के बुनियादी तत्व, AI के निर्माण खंड और रक्षात्मक-LLM सुरक्षा उपाय सभी स्वतंत्र रूप से परीक्षित, प्लग-एंड-प्ले मॉड्यूल के रूप में मौजूद हैं। यही वजह है कि एक छोटी टीम भी बड़े पैमाने की टीमों जैसी गति से उत्पाद-स्तरीय सिस्टम विकसित कर पाती है, और ऐसा करते हुए उस दोहराव के कर्ज से बच जाती है जो आमतौर पर इसके बाद जमा होता है।

क्या है नवीन

- संपूर्ण सिस्टम में डिकपलिंग अनुशासन (**CONST-051**): सबमॉड्यूल्स को समान कोडबेस के रूप में माना जाता है, जिनमें उपभोक्ता-विशिष्ट विवरण कभी शामिल नहीं होते।
- समर्पित AI-प्रिमिटिव परत (**RAG, वेक्टरडीबी, एम्बेडिंग्स, MCP, टूलस्कीमा, एजेंटिक, प्लानिंग, LLMops**) पुनःप्रयोग योग्य मॉड्यूल के रूप में।
- रक्षात्मक-LLM सुरक्षा समूह (**रेडटीम, नॉर्मलाइज**) प्रतिकूल परिस्थितियों में मज़बूती के लिए।
- समान नियमों पर आधारित समानांतर **Go + Kotlin Multiplatform** मॉड्यूल सेट्स।

चुनौतियाँ और समाधान

- दर्जनों मॉड्यूल्स में कपलिंग सड़न से बचाव: संविधान के डिक्पलिंग अनुबंध और उपभोक्ता-विशिष्ट रनटाइम इंजेक्शन से हल।
- कई मॉड्यूल्स को सुसंगत और परीक्षित रखना: साझा परंपरा (प्रति-मॉड्यूल टेस्ट/डॉक्स/चुनौतियाँ) और HelixConstitution गवर्नेंस बैकबोन से हल।
- क्रॉस-प्लेटफॉर्म पहुँच: मुख्य मॉड्यूल्स का Kotlin Multiplatform मिरर से हल।

तकनीकी स्टैक (क्यों + कैसे)

- **Go** — अधिकांश मॉड्यूल्स (digital.vasic.*)।
- **Kotlin Multiplatform** — क्रॉस-प्लेटफॉर्म मिरर मॉड्यूल्स (ऑथ/डेटाबेस/सुरक्षा/यूआई/कंकरेंसी/रेटलिमिटर-KMP)।
- **Redis / PostgreSQL / SQLite** — कैश, डेटाबेस, स्टोरेज प्रिमिटिव्स।
- **Prometheus / OpenTelemetry** — ऑब्जर्वेबिलिटी मॉड्यूल।
- **WebSocket / HTTP/3 (quic-go) / mDNS** — नेटवर्किंग मॉड्यूल्स।
- **वेक्टर डीबी / embeddings / RAG / MCP** — AI-प्रिमिटिव मॉड्यूल्स।
- **YAML** — रेडटीम प्रतिकूल फिक्स्चर और कॉन्फिगरेशन।

असत्यापित / कार्य प्रगति पर: कई संगठनात्मक रिपॉज़िटरीज़ को “स्कैफोल्ड / डब्ल्यूआईपी” के रूप में चिह्नित किया गया है (जैसे PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions)। इन्हें प्रारंभिक चरण/स्कैफोल्ड के रूप में प्रस्तुत करें, शिप नहीं किया गया।